

# How the whole thing works

A plain-language walkthrough: what we are pricing, what each model assumes, what every parameter captures, what the PINN loss function is, how it is minimised, and how the results are tested.

## 1 The problem

A European call option gives the right to buy an asset at a fixed strike  $K$  on a fixed date. Its price  $C$  splits into two parts:

$$C = \max(S - K, 0) + \text{time value}$$

(intrinsic value: arithmetic)

(what the model decides)

Every model agrees on the intrinsic value — it is arithmetic. All the disagreement lives in the time value, which is the market paying for the chance that the price moves before expiry. How much that chance is worth depends entirely on what you assume about volatility. That assumption is the model.

## 2 The three models, in one sentence each

| model         | assumption about volatility                            | free numbers        |
|---------------|--------------------------------------------------------|---------------------|
| Black-Scholes | constant, one number forever                           | 1 (or 1 per expiry) |
| Single Heston | random, pulled back to one long-run level              | 5                   |
| Double Heston | random, two independent components: one fast, one slow | 10                  |

That is the entire hierarchy. Each step adds a way for volatility to behave that the previous model could not express.

## 3 What goes wrong with each, and why it matters

Black-Scholes assumes one volatility for all strikes. But the market charges more for low-strike puts (crash protection) than for high-strike calls. Plot the market's implied volatility against strike and you get a "smile" or "skew". Black-Scholes must draw a horizontal line through it. On the real IWM surface that costs it 5.39 volatility points of error.

Single Heston lets volatility move randomly and pulls it back toward a long-run level at a single speed  $\kappa$ . One speed means one timescale. If today's volatility is elevated, Single Heston must decay it back at one rate, so it cannot simultaneously match what short-dated options say (volatility falls fast) and what long-dated options say (volatility stays elevated). On our controlled benchmark that mismatch peaks at 0.85 volatility points around 7 days, falls to zero near 110 days, then flips sign.

Double Heston splits volatility into two pieces with their own speeds — one that decays in about 24 days and one that takes 267 days. Short-dated options are priced mostly by the fast piece, long-dated ones mostly by the slow piece, and the model can match both ends at once.

## 4 The key idea in one experiment

Take two situations with exactly the same volatility today, 20%. In the first, the volatility sits in the fast component; in the second, in the slow one. Total variance is 0.04 in both cases.

| maturity | fast-heavy ATM vol | slow-heavy ATM vol | call price, fast vs slow |
|----------|--------------------|--------------------|--------------------------|
| 30 days  | 19.8%              | 22.1%              | 2.29 vs 2.55             |
| 90 days  | 20.0%              | 23.8%              | 3.96 vs 4.71             |
| 1 year   | 21.1%              | 24.8%              | 8.42 vs 9.87             |
| 2 years  | 22.1%              | 24.7%              | 12.42 vs 13.87           |

Same volatility today, prices differ by up to 12% at every maturity — because variance sitting in the fast component evaporates 11.3 times faster. A one-factor model cannot produce both columns from the same starting volatility. That is the entire case for Double Heston.

## 5 What each of the ten parameters actually controls

Each variance factor has five parameters. Double Heston has two factors, so ten. Here is what each one does to the price, in words.

| symbol   | name              | what it captures                                         | raise it and ...                      |
|----------|-------------------|----------------------------------------------------------|---------------------------------------|
| $\kappa$ | mean reversion    | how fast volatility forgets today and returns to normal  | today's vol matters less              |
| $\theta$ | long-run variance | the level volatility drifts back to                      | long-dated options cost more          |
| $\sigma$ | vol-of-vol        | how uncertain the future volatility itself is            | the smile gets more curved            |
| $\rho$   | correlation       | whether vol rises when price falls (the leverage effect) | the skew tilts (neg = crash-fear)     |
| $v_0$    | initial variance  | how volatile the asset is right now                      | everything costs more, short end most |

So kappa and theta shape the TERM STRUCTURE (how price varies with maturity), while sigma and rho shape the SMILE (how price varies with strike), and  $v_0$  sets the overall level today.

## 6 The published values we use, and what they say

|             | $\kappa$ | $\theta$ | $\sigma$ | $\rho$  | $v_0$  | reading                          |
|-------------|----------|----------|----------|---------|--------|----------------------------------|
| slow factor | 0.9491   | 0.0257   | 0.0517   | +0.7009 | 0.0003 | calm, persistent, 267-day memory |
| fast factor | 10.7526  | 0.0330   | 0.3613   | -0.8916 | 0.0252 | jumpy, 24-day memory, crash-fear |

Two things stand out. The speeds differ by 11.3 times, which is what makes the two factors genuinely different rather than redundant. And the correlations have OPPOSITE SIGNS: the fast factor is strongly negative (-0.89, the usual crash-fear tilt), while the slow one is positive (+0.70). A single factor must pick one number, so it cannot produce a skew that changes character with maturity. This is why the model must be written with four separate shocks rather than the restricted form.

## 7 Why "hard-coded" and what the one fitted number does

In several experiments we deliberately do NOT fit these ten numbers — we use the published values and ask what the STRUCTURE alone is worth. The only number ever fitted in that setting is a single level scale  $s$ , which multiplies the variances:

$$\theta_i \mapsto s \theta_i, \quad v_{i,0} \mapsto s v_{i,0}, \quad \sigma_i \mapsto \sqrt{s} \sigma_i, \quad s \in [0.7, 3.2]$$

This shifts the overall volatility level up or down without touching any timescale, correlation or shape. It is there so that a parameter set fitted to 1990s equities can be tried on a market trading at a different volatility level, without secretly refitting the structure.

Result worth knowing: with the structure frozen this way, Double Heston beats Single Heston by only about 0.1-0.3 volatility points on real surfaces. When all ten parameters are fitted to the surface, the same model beats Single Heston by 3.2 times on held-out quotes. The two-factor structure is real, but it only pays once the parameters are allowed to describe the market in front of you.

## 8 Why a neural network at all

Pricing one Double Heston option exactly means numerically integrating a complex-valued function. That is accurate but slow, and calibration needs hundreds of thousands of such prices. The PINN is trained to reproduce the exact answer in a single fast forward pass. It is a SURROGATE for the model, not a new model: the target it is trained against is exact Double Heston.

## 9 The design that makes it work: prior plus correction

A naive network would try to predict the price directly and would have to learn the payoff kink, the no-arbitrage bounds and the shape of the entire surface from scratch. Instead the network is given a head start. First we compute an analytic baseline volatility — the average variance the model expects over the life of the option:

$$\bar{v}(\tau) = \sum_{i \in \{s, f\}} \left[ \theta_i + (v_i - \theta_i) \frac{1 - e^{-\kappa_i \tau}}{\kappa_i \tau} \right]$$

This formula already knows the two-timescale behaviour: the bracket interpolates between today's variance  $v_i$  at short maturity and the long-run level  $\theta_i$  at long maturity, at each factor's own speed. It is not exact, because it ignores the smile and the correlation effects. The network only has to supply the correction:

$$\sigma_{imp} = \sqrt{\bar{v}} \times e^{1.8 \tanh(\text{network output})}$$

The tanh bounds the correction, so the network can scale the baseline volatility by at most a factor of about 6 either way and can never produce nonsense. The resulting volatility is then fed through the Black formula to get the price. Two properties come for free: at expiry the price is exactly the payoff, and the price can never violate its basic bounds. The network cannot break them even if it wanted to.

## 10 What the network sees

Twelve raw inputs: log-moneyness  $x = \log(F/K)$ , the two variances, time to maturity, and the eight structural parameters. These are expanded into 24 fixed financial features — things like scaled log-maturity, a standardised moneyness,  $\tanh(\kappa \tau)$  which says how far through the factor's memory we are, a vol-of-vol measure, and skew terms. None of these are learned; they are hand-built so the network starts from quantities that already mean something financially.

## 11 The body of the network

|             | original (v4)                   | improved (v5)                               |
|-------------|---------------------------------|---------------------------------------------|
| layers      | 5 layers, width 256, plain tanh | 5 gated blocks, width 256                   |
| connections | none beyond layer to layer      | gated skip paths through every block        |
| activation  | fixed tanh                      | $\tanh(a_i z)$ with $a_i$ learned per block |
| parameters  | 269,825                         | 282,632 (+4.7%)                             |

The gated block keeps two encodings U and V of the input features available at every depth and lets the network mix between them, so information and gradients have a short path from input to output. That is the single biggest architectural gain we measured.

## 12 The loss function, term by term

Training minimises one number. It is built from four pieces, each measuring a different kind of being wrong.

| term        | what it measures                                       | why it is there                               |
|-------------|--------------------------------------------------------|-----------------------------------------------|
| $L_{price}$ | squared price error against the exact Fourier price    | the main job: reproduce the model             |
| $L_{IV}$    | squared implied-volatility error against the exact one | price error alone under-weights cheap options |
| $L_{PDE}$   | how badly the output violates the pricing equation     | physics: forces a valid solution everywhere   |
| $L_{conv}$  | penalty when the surface bends the wrong way           | blocks butterfly arbitrage                    |

The first two are supervised: they compare against 100,000 exactly computed teacher prices. The third is unsupervised: at 18,000 extra "collocation" points, with no labels at all, automatic differentiation computes the derivatives of the network output and checks whether they satisfy the two-factor Heston partial differential equation. That is the "physics-informed" part — the network is told the law its answer must obey, not just examples of correct answers.

The four pieces have wildly different natural sizes: prices are around  $1e-5$ , implied vols around  $1e-3$ . Adding them raw would let one term dominate. So each is divided by the square of the accuracy we actually require of it — its acceptance tolerance:

$$L = \frac{L_{price}}{(2 \times 10^{-5})^2} + \frac{L_{IV}}{(0.002)^2} + 0.1 \cdot \frac{L_{PDE}}{(0.01)^2} + 0.1 \cdot L_{conv}$$

This normalisation was itself a bug fix. An earlier version divided the price term by  $1e-3$  instead of  $2e-5$ , which made it only 1.5-2% of the total loss. The network was underfitting the thing we cared about most, and no amount of extra training fixed it. Rescaling each term by its own acceptance tolerance cut the error by an order of magnitude.

## 13 How it is minimised

- Adam for 40,000 steps. Each step draws a random batch of 512 teacher points and 128 physics points, computes the loss, backpropagates, and nudges all 282,632 weights downhill. The learning rate follows a cosine schedule from  $1e-3$  down to  $1e-5$ , so it explores early and settles late.
- Then L-BFGS for 300 steps on a fixed large batch. This is a second-order method: it builds an approximation of the curvature and takes much better-aimed steps. It is too memory-hungry for the whole run but excellent for polishing the last digits.
- Two random seeds are trained independently and their prices averaged, which removes some of the luck of initialisation.

The improved version adds two mechanisms during training:

- Gradient balancing. Every 100 steps we measure how large a gradient each loss term produces. If the physics term is pushing 100 times harder than the price term, it silently dominates. The weights are rebalanced toward the ratio of gradient norms, smoothed 90/10 so they cannot oscillate.
- Residual-adaptive sampling. Every 2,000 steps half of the physics points are redrawn, with probability proportional to how badly the equation is currently violated there. The network gets more practice exactly where it is failing. The budget never grows; the points just move.

## 14 How we know the training worked

Four acceptance gates were fixed before any training: price RMSE below  $2e-5$ , 95th percentile below  $5e-5$ , worst case below  $2e-4$ , and implied-vol RMSE below 0.2 volatility points. A candidate architecture is only accepted if all four pass on a development set. The final network is then measured once on a test set of 16,384 points that had never been opened.

| measured on the untouched test set | original v4 | improved v5    |
|------------------------------------|-------------|----------------|
| typical price error                | 1.06e-5     | 6.61e-6        |
| worst price error                  | 1.52e-4     | 5.94e-5        |
| implied-vol error (vol points)     | 0.1175      | 0.0738         |
| physics residual                   | 0.0309      | 0.0558 (worse) |

So the improved network is about 1.6 times more accurate as a solver, and its worst case is 2.6 times better — at the cost of 4.7% more parameters, 1.7 times slower inference, and a worse physics residual, because gradient balancing shifted weight toward the supervised terms.

## 15 How the models are fitted to real market prices

Separately from the PINN, the models are calibrated to observed option prices. We minimise a vega-weighted price error:

$$\min \left\langle \left( \frac{C_{\text{model}} - C_{\text{market}}}{\text{vega}} \right)^2 \right\rangle$$

Dividing by vega — the sensitivity of price to volatility — converts a price error into approximately a volatility error. Without it, the fit would obsess over expensive at-the-money options and ignore the wings, where the interesting structure lives. The optimiser is differential evolution (a global search that does not need a good starting guess) followed by 12 local least-squares refinements from different starting points, because these objectives have many local minima. Every start is kept and reported.

## 16 How the comparison is kept honest

- Held-out quotes. Models are fitted on half the surface, in a checkerboard pattern, and scored on the other half, which they never saw. A model that merely has more parameters gains nothing.
- Rules frozen first. Filters, date rules, endpoints and statistical tests are written down and hashed before any data is downloaded. Data selection uses market structure only, never model error.
- A win must clear two bars: a one-sided Wilcoxon test with  $p < 0.05$ , AND a bootstrap confidence interval whose lower bound is above zero, clustered so correlated dates cannot fake significance.
- Failures are kept. Flawed runs, superseded results and unfavourable outcomes stay in the repository and in the reports.

## 17 The results, briefly

On a real surface chosen in advance for its two-timescale structure (IWM), with every model given its strongest fair calibration and scored on 707 quotes it never saw:

| model                             | free parameters | held-out error (vol points) |
|-----------------------------------|-----------------|-----------------------------|
| Black-Scholes, one vol per expiry | 22              | 5.39                        |
| Single Heston                     | 5               | 1.89                        |
| Double Heston                     | 10              | 0.58                        |

Double Heston is 3.2 times better than Single Heston, and Black-Scholes is worst despite having the most free parameters — extra volatility levels cannot make a smile.

On crypto options with per-date calibration, Double Heston beat Single Heston significantly on BTC and ETH (about 7 expiries per day) but not on SOL, XRP or HYPE (about 4 expiries per day). The advantage is real but needs enough liquid expiries to identify the second factor.

With the published parameters frozen, the advantage shrinks to 0.0-0.3 volatility points on every real surface tested. And improving the PINN itself moved real market error by 0.003 volatility points out of 5.8, because network error is about 1% of model error.

## 18 The mental model to take away

| layer                 | what it fixes                                  | what it costs                         |
|-----------------------|------------------------------------------------|---------------------------------------|
| Black-Scholes         | nothing; it is the reference shape             | 1 number, no smile, no term structure |
| + one variance factor | smile and a single decay speed                 | 5 numbers                             |
| + second factor       | short and long horizons independently          | 10 numbers, needs rich data           |
| + PINN surrogate      | speed: one forward pass instead of an integral | approximation error 7e-6              |
| + residual head       | whatever the model still cannot say            | no longer a pure model                |

The single most useful thing this project established: the gap between a fixed-parameter model and the market is roughly a hundred times larger than the error of the neural surrogate. So improving the network is a speed and accuracy win for the solver, but it is not a route to better market prices. Better market prices come from letting the parameters describe the market, or from admitting a correction term and labelling it honestly.

Companion documents: MODEL\_EQUATIONS\_REFERENCE.pdf for every formula and range; the per-experiment REPORT.md files for the full results and their caveats.